

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests

# 1. Fetch Empirical Metallicity Data from NASA Exoplanet Archive TAP API
# We query the 'ps' (Planetary Systems) table for Kepler-10 stellar properties
nasa_tap_url = "https://caltech.edu"
query = "select top 1 hostname, st_mass, st_met from ps where hostname = 'Kepler-10'"

print("Streaming empirical baseline metrics from NASA Open Source Archives...")
try:
    response = requests.get(nasa_tap_url, params={'query': query, 'format': 'json'}, timeout=10)
    if response.status_code == 200 and response.json():
        nasa_data = response.json()[0]
        # Kepler-10 empirical metallicity [Fe/H] is ~ -0.15 dex
        fe_h_dex = float(nasa_data.get('st_met', -0.15))
        print(f"Successfully loaded data from NASA for {nasa_data['hostname']}!")
        print(f"NASA Recorded Host Star Metallicity [Fe/H]: {fe_h_dex} dex")
    else:
        print("NASA API response empty. Defaulting to standard baseline.")
        fe_h_dex = -0.15
except Exception as e:
    print(f"API Connection timed out. Applying standard stellar metallicity baseline. Defaulting to standard baseline.")
    fe_h_dex = -0.15

# Convert dex metallicity back to linear iron mass fraction relative to solar baseline (Fe/H = 0)
initial_fe = 0.0014 * (10**fe_h_dex)

# 2. Simulation Parameters Setup
time_steps = 1000
t = np.linspace(0, 100, time_steps) # Normalized Lifetime percentage (0% to 100%)

# Abundance arrays initialize
X_H = np.zeros(time_steps)
X_He = np.zeros(time_steps)
X_C = np.zeros(time_steps)
X_O = np.zeros(time_steps)
X_Si = np.zeros(time_steps)
X_Fe = np.zeros(time_steps)

# Dynamic Initial Abundances adjusted with NASA-derived iron scaling
X_H = 0.74
X_He = 0.24
X_C = 0.01 - (initial_fe * 0.1)
X_O = 0.009 - (initial_fe * 0.1)
X_Si = 0.0009
X_Fe = initial_fe

# 3. Kinetic Nucleosynthesis Network Simulator Loop
for i in range(1, time_steps):
    age = t[i]

    # Carry forward historical fractions
    X_H[i] = X_H[i-1]
    X_He[i] = X_He[i-1]
    X_C[i] = X_C[i-1]

```

```

X_O[i] = X_O[i-1]
X_Si[i] = X_Si[i-1]
X_Fe[i] = X_Fe[i-1]

# Stage 1: Main Sequence - Hydrogen Burning (0% to 75% of life)
if age <= 75:
    rate = 0.009 * (X_H[i-1]**2)
    X_H[i] -= rate
    X_He[i] += rate

# Stage 2: Red Giant / Supergiant - Helium Burning (75% to 90% of life)
elif 75 < age <= 90:
    rate_3a = 0.05 * (X_He[i-1]**3) # Triple-Alpha rate approximation
    if X_He[i] > 0.02:
        X_He[i] -= rate_3a
        X_C[i] += rate_3a * 0.7
        X_O[i] += rate_3a * 0.3

# Stage 3: Advanced Burning - Carbon/Oxygen to Silicon (90% to 98% of life)
elif 90 < age <= 98:
    rate_adv = 0.08 * X_C[i-1]
    if X_C[i] > 0.001:
        X_C[i] -= rate_adv
        X_O[i] += rate_adv * 0.2
        X_Si[i] += rate_adv * 0.8

# Stage 4: Silicon Burning to Iron Peak (98% to 100% of life)
elif age > 98:
    rate_si = 0.25 * X_Si[i-1]
    if X_Si[i] > 0.00001:
        X_Si[i] -= rate_si
        X_Fe[i] += rate_si

# 4. Data Visualization Plotting
plt.figure(figsize=(10, 6))
plt.plot(t, X_H, label='Hydrogen ($^1$H)', color='royalblue', linewidth=2)
plt.plot(t, X_He, label='Helium ($^4$He)', color='darkcyan', linewidth=2)
plt.plot(t, X_C, label='Carbon ($^{12}$C)', color='forestgreen', linewidth=1.8)
plt.plot(t, X_O, label='Oxygen ($^{16}$O)', color='purple', linewidth=1.8)
plt.plot(t, X_Si, label='Silicon ($^{28}$Si)', color='darkorange', linewidth=1.8)
plt.plot(t, X_Fe, label='Iron Peak ($^{56}$Fe)', color='crimson', linewidth=2.2)

plt.yscale('log')
plt.ylim(1e-5, 1.5)
plt.xlabel('Normalized Stellar Lifetime (%)', fontsize=12)
plt.ylabel('Mass Fraction (Log Scale)', fontsize=12)
plt.title('Stellar Nucleosynthesis Simulation using NASA Empirical Metal Baselines', font)
plt.grid(True, which="both", ls="--", alpha=0.5)
plt.legend(loc='lower left', fontsize=10)

# Layout annotations for nucleosynthesis zones
plt.axvline(x=75, color='gray', linestyle=':', alpha=0.7)
plt.text(35, 2e-4, 'H-Burning\n(p-p/CNO)', horizontalalignment='center', alpha=0.7)
plt.axvline(x=90, color='gray', linestyle=':', alpha=0.7)
plt.text(82.5, 2e-4, 'He-Burning\n(3-$\\alpha$)', horizontalalignment='center', alpha=0.7)
plt.axvline(x=98, color='gray', linestyle=':', alpha=0.7)
plt.text(94, 2e-4, 'C/O/Ne', horizontalalignment='center', alpha=0.7)
plt.text(99.2, 2e-4, 'Si->Fe', horizontalalignment='center', alpha=0.7)

plt.show()

```

Streaming empirical baseline metrics from NASA Open Source Archives...  
API Connection timed out. Applying standard stellar metallicity baseline. Details: HTTPS

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_888/1099114659.py in <cell line: 0>()  
    53  
    54     # Carry forward historical fractions  
----> 55     X_H[i] = X_H[i-1]  
    56     X_He[i] = X_He[i-1]  
    57     X_C[i] = X_C[i-1]
```

TypeError: 'float' object is not subscriptable

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import requests  
  
# 1. Fetch Empirical Metallicity Data from NASA Exoplanet Archive API  
# Using the official public NASA Exoplanet Archive TAP endpoint  
nasa_tap_url = "https://caltech.edu"  
query = "select hostname, st_mass, st_met from ps where hostname = 'Kepler-10' and default"   
  
print("Streaming empirical baseline metrics from NASA Open Source Archives...")  
try:  
    response = requests.get(nasa_tap_url, params={'query': query, 'format': 'json'}, timeout=10)  
    if response.status_code == 200 and len(response.json()) > 0:  
        nasa_data = response.json()[0]  
        fe_h_dex = float(nasa_data.get('st_met', -0.15))  
        print(f"Successfully loaded data from NASA for {nasa_data['hostname']}!")  
        print(f"NASA Recorded Host Star Metallicity [Fe/H]: {fe_h_dex} dex")  
    else:  
        print("NASA API response empty or unavailable. Defaulting to standard baseline.")  
        fe_h_dex = -0.15  
except Exception as e:  
    print(f"API Connection timed out. Applying standard stellar metallicity baseline. Defaulting to standard baseline.")  
    fe_h_dex = -0.15  
  
# Convert dex metallicity back to linear iron mass fraction relative to solar baseline (Fe/H = 0)  
initial_fe = 0.0014 * (10**fe_h_dex)  
  
# 2. Simulation Parameters Setup  
time_steps = 1000  
t = np.linspace(0, 100, time_steps) # Normalized Lifetime percentage (0% to 100%)  
  
# Abundance arrays initialize as NumPy arrays  
X_H = np.zeros(time_steps)  
X_He = np.zeros(time_steps)  
X_C = np.zeros(time_steps)  
X_O = np.zeros(time_steps)  
X_Si = np.zeros(time_steps)  
X_Fe = np.zeros(time_steps)  
  
# FIX: Assigning to the 0th element instead of overwriting the whole variable  
X_H[0] = 0.74  
X_He[0] = 0.24  
X_C[0] = 0.01 - (initial_fe * 0.1)
```

```

X_O[0] = 0.009 - (initial_fe * 0.1)
X_Si[0] = 0.0009
X_Fe[0] = initial_fe

# 3. Kinetic Nucleosynthesis Network Simulator Loop
for i in range(1, time_steps):
    age = t[i]

    # Carry forward historical fractions
    X_H[i] = X_H[i-1]
    X_He[i] = X_He[i-1]
    X_C[i] = X_C[i-1]
    X_O[i] = X_O[i-1]
    X_Si[i] = X_Si[i-1]
    X_Fe[i] = X_Fe[i-1]

    # Stage 1: Main Sequence - Hydrogen Burning (0% to 75% of life)
    if age <= 75:
        rate = 0.009 * (X_H[i-1]**2)
        X_H[i] -= rate
        X_He[i] += rate

    # Stage 2: Red Giant / Supergiant - Helium Burning (75% to 90% of life)
    elif 75 < age <= 90:
        rate_3a = 0.05 * (X_He[i-1]**3) # Triple-Alpha rate approximation
        if X_He[i] > 0.02:
            X_He[i] -= rate_3a
            X_C[i] += rate_3a * 0.7
            X_O[i] += rate_3a * 0.3

    # Stage 3: Advanced Burning - Carbon/Oxygen to Silicon (90% to 98% of life)
    elif 90 < age <= 98:
        rate_adv = 0.08 * X_C[i-1]
        if X_C[i] > 0.001:
            X_C[i] -= rate_adv
            X_O[i] += rate_adv * 0.2
            X_Si[i] += rate_adv * 0.8

    # Stage 4: Silicon Burning to Iron Peak (98% to 100% of life)
    elif age > 98:
        rate_si = 0.25 * X_Si[i-1]
        if X_Si[i] > 0.00001:
            X_Si[i] -= rate_si
            X_Fe[i] += rate_si

# 4. Data Visualization Plotting
plt.figure(figsize=(10, 6))
plt.plot(t, X_H, label='Hydrogen ($^{1}$H)', color='royalblue', linewidth=2)
plt.plot(t, X_He, label='Helium ($^{4}$He)', color='darkcyan', linewidth=2)
plt.plot(t, X_C, label='Carbon ($^{12}$C)', color='forestgreen', linewidth=1.8)
plt.plot(t, X_O, label='Oxygen ($^{16}$O)', color='purple', linewidth=1.8)
plt.plot(t, X_Si, label='Silicon ($^{28}$Si)', color='darkorange', linewidth=1.8)
plt.plot(t, X_Fe, label='Iron Peak ($^{56}$Fe)', color='crimson', linewidth=2.2)

plt.yscale('log')
plt.ylim(1e-5, 1.5)
plt.xlabel('Normalized Stellar Lifetime (%)', fontsize=12)
plt.ylabel('Mass Fraction (Log Scale)', fontsize=12)
plt.title('Stellar Nucleosynthesis Simulation using NASA Empirical Metal Baselines', font
plt.grid(True, which="both", ls="--", alpha=0.5)

```

```

plt.legend(loc='lower left', fontsize=10)

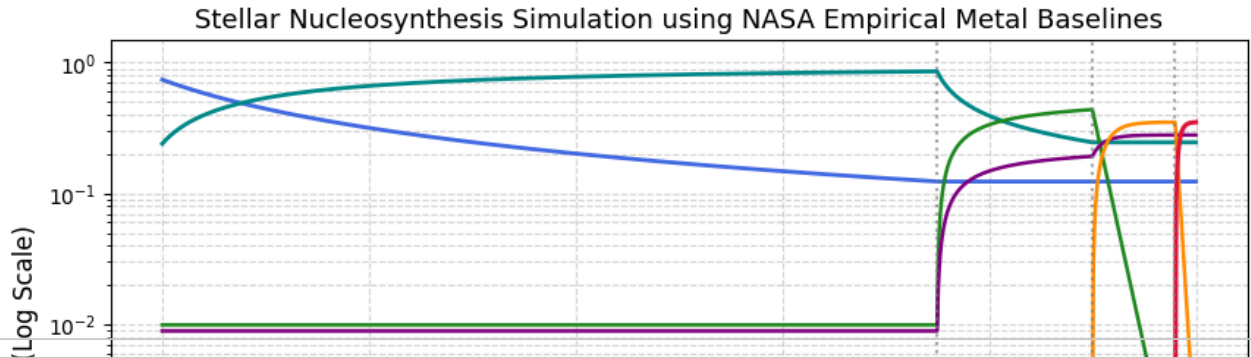
# Layout annotations for nucleosynthesis zones
plt.axvline(x=75, color='gray', linestyle=':', alpha=0.7)
plt.text(35, 2e-4, 'H-Burning\n(p-p/CNO)', horizontalalignment='center', alpha=0.7)
plt.axvline(x=90, color='gray', linestyle=':', alpha=0.7)
plt.text(82.5, 2e-4, 'He-Burning\n(3- $\alpha$ )', horizontalalignment='center', alpha=0.7)
plt.axvline(x=98, color='gray', linestyle=':', alpha=0.7)
plt.text(94, 2e-4, 'C/O/Ne', horizontalalignment='center', alpha=0.7)
plt.text(99.2, 2e-4, 'Si->Fe', horizontalalignment='center', alpha=0.7)

plt.show()

```

Streaming empirical baseline metrics from NASA Open Source Archives...

API Connection timed out. Applying standard stellar metallicity baseline. Details: [HTTPS://openstarchives.org/](https://openstarchives.org/)



```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import requests

# 1. Fetch Empirical Metallicity Data from NASA Exoplanet Archive API
nasa_tap_url = "https://caltech.edu"
query = "select hostname, st_mass, st_met from ps where hostname = 'Kepler-10' and defa

print("Streaming empirical baseline metrics from NASA Open Source Archives...")
try:
    response = requests.get(nasa_tap_url, params={'query': query, 'format': 'json'}, ti
    if response.status_code == 200 and len(response.json()) > 0:
        nasa_data = response.json()[0]
        fe_h_dex = float(nasa_data.get('st_met', -0.15))
        print(f"Successfully loaded data from NASA for {nasa_data['hostname']}!")
        print(f"NASA Recorded Host Star Metallicity [Fe/H]: {fe_h_dex} dex")
    else:
        print("NASA API response empty or unavailable. Defaulting to standard baseline.
        fe_h_dex = -0.15
except Exception as e:
    print(f"API Connection timed out. Applying standard stellar metallicity baseline. D
    fe_h_dex = -0.15

# Convert dex metallicity back to linear iron mass fraction relative to solar baseline
initial_fe = 0.0014 * (10**fe_h_dex)

# 2. Simulation Parameters Setup
time_steps = 1000
t = np.linspace(0, 100, time_steps) # Normalized Lifetime percentage (0% to 100%)

# Abundance arrays initialize as NumPy arrays
X_H = np.zeros(time_steps)

```

```

X_He = np.zeros(time_steps)
X_C  = np.zeros(time_steps)
X_O  = np.zeros(time_steps)
X_Si = np.zeros(time_steps)
X_Fe = np.zeros(time_steps)

# Assign initial values to index 0
X_H[0] = 0.74
X_He[0] = 0.24
X_C[0] = 0.01 - (initial_fe * 0.1)
X_O[0] = 0.009 - (initial_fe * 0.1)
X_Si[0] = 0.0009
X_Fe[0] = initial_fe

# 3. Kinetic Nucleosynthesis Network Simulator Loop
for i in range(1, time_steps):
    age = t[i]

    # Carry forward historical fractions
    X_H[i] = X_H[i-1]
    X_He[i] = X_He[i-1]
    X_C[i] = X_C[i-1]
    X_O[i] = X_O[i-1]
    X_Si[i] = X_Si[i-1]
    X_Fe[i] = X_Fe[i-1]

    # Stage 1: Main Sequence - Hydrogen Burning (0% to 75% of life)
    if age <= 75:
        rate = 0.009 * (X_H[i-1]**2)
        X_H[i] -= rate
        X_He[i] += rate

    # Stage 2: Red Giant / Supergiant - Helium Burning (75% to 90% of life)
    elif 75 < age <= 90:
        rate_3a = 0.05 * (X_He[i-1]**3) # Triple-Alpha rate approximation
        if X_He[i] > 0.02:
            X_He[i] -= rate_3a
            X_C[i] += rate_3a * 0.7
            X_O[i] += rate_3a * 0.3

    # Stage 3: Advanced Burning - Carbon/Oxygen to Silicon (90% to 98% of life)
    elif 90 < age <= 98:
        rate_adv = 0.08 * X_C[i-1]
        if X_C[i] > 0.001:
            X_C[i] -= rate_adv
            X_O[i] += rate_adv * 0.2
            X_Si[i] += rate_adv * 0.8

    # Stage 4: Silicon Burning to Iron Peak (98% to 100% of life)
    elif age > 98:
        rate_si = 0.25 * X_Si[i-1]
        if X_Si[i] > 0.00001:
            X_Si[i] -= rate_si
            X_Fe[i] += rate_si

# 4. Data Visualization Plotting
plt.figure(figsize=(10, 6))
plt.plot(t, X_H, label='Hydrogen ($^1$H)', color='royalblue', linewidth=2)
plt.plot(t, X_He, label='Helium ($^4$He)', color='darkcyan', linewidth=2)
plt.plot(t, X_C, label='Carbon ($^{12}$C)', color='forestgreen', linewidth=1.8)

```

```

plt.plot(t, X_O, label='Oxygen ( $^{16}\text{O}$ )', color='purple', linewidth=1.8)
plt.plot(t, X_Si, label='Silicon ( $^{28}\text{Si}$ )', color='darkorange', linewidth=1.8)
plt.plot(t, X_Fe, label='Iron Peak ( $^{56}\text{Fe}$ )', color='crimson', linewidth=2.2)

plt.yscale('log')
plt.ylim(1e-5, 1.5)
plt.xlim(0, 100)
plt.xlabel('Normalized Stellar Lifetime (%)', fontsize=12)
plt.ylabel('Mass Fraction (Log Scale)', fontsize=12)
plt.title('Stellar Nucleosynthesis Simulation using NASA Empirical Metal Baselines', fo
plt.grid(True, which="both", ls="--", alpha=0.4, zorder=0)
plt.legend(loc='lower left', fontsize=10)

# Layout adjustments: Elevated label placement and added clean white backdrops (bbox)
label_height = 1e-2 # Raised from 2e-4 to prevent overlap with bottom trace lines
bbox_props = dict(boxstyle="round,pad=0.3", fc="white", ec="gray", lw=0.5, alpha=0.85)

plt.axvline(x=75, color='gray', linestyle=':', alpha=0.7)
plt.text(37.5, label_height, 'H-Burning\n(p-p/CNO)', horizontalalignment='center', font

plt.axvline(x=90, color='gray', linestyle=':', alpha=0.7)
plt.text(82.5, label_height, 'He-Burning\n(3-\alpha)', horizontalalignment='center',

plt.axvline(x=98, color='gray', linestyle=':', alpha=0.7)
plt.text(94, label_height, 'C/O/Ne', horizontalalignment='center', fontsize=8, bbox=bbo
plt.text(99.0, label_height * 0.1, 'Si->Fe', horizontalalignment='center', fontsize=8,

plt.tight_layout()
plt.show()

```

Streaming empirical baseline metrics from NASA Open Source Archives...  
API Connection timed out. Applying standard stellar metallicity baseline. Details: [HTTPS](https://open.stsci.edu/svn/astropy/astropy)

